

REAL-TIME PATIENT MONITORING SYSTEM

Development on Raspberry Pi 5

From Hardware Struggles to a Cloud-Connected IoT Prototype

Prepared by: Masrur Jamil Prochchhod

Institution: Hochschule Hamm-Lippstadt (HSHL)

Platform: Raspberry Pi 5 | Python 3 | Blynk IoT

1. Executive Summary

This report provides a complete chronological account of the development of an IoT-based Real-Time Patient Monitoring System, implemented on a Raspberry Pi 5 as part of the HSHL internship project. The work covers everything from the initial hardware setup struggles through to a functioning multi-threaded software prototype with local hardware output and cloud-based remote monitoring.

The system monitors three patient vital parameters — Heart Rate, Body Temperature, and Physical Acceleration — generated via a programmatic simulation engine. These values are displayed locally on a 16x2 LCD panel, trigger an auditory buzzer alarm on threshold breaches, and are visualized in real-time through a Python-based live plotting interface. The system is also synchronized to a mobile dashboard via Blynk IoT for remote monitoring.

The journey was far from linear. Before any software could be written, significant time was spent battling repeated boot failures on the Raspberry Pi 5 caused by a faulty SD card — a phase that consumed days of troubleshooting and waiting. Once the hardware was stabilized, the development phase introduced its own set of challenges: display server incompatibilities on the new Wayland platform, file-level race conditions from concurrent CSV access, and memory management issues in the live animation engine — all of which were systematically identified and resolved.

2. Project Goals & Scope

The core objectives of this project were defined as follows:

- Build a working prototype of a patient vital signs monitoring station on Raspberry Pi 5
- Simulate realistic biomedical sensor data (heart rate, temperature, acceleration) programmatically
- Integrate physical output hardware: a 16x2 LCD display and an auditory alarm buzzer
- Develop a real-time live plotting interface for clinical visual monitoring
- Implement threshold-based alerting with visual and auditory warnings
- Connect the system to a cloud IoT dashboard via Blynk for remote mobile monitoring

Scope note: Physical biomedical sensors (e.g., MAX30102, DS18B20, MPU6050) were not wired up for live data input in this prototype phase. Sensor readings were instead generated programmatically via a dedicated simulation script (`sim.py`). The real hardware components integrated were the LCD display and the buzzer — connected directly to the Raspberry Pi 5 GPIO pins.

3. Hardware Architecture & Setup

The physical hardware platform for this project centres on the Raspberry Pi 5 single-board computer, which provides the processing, GPIO interfacing, and network connectivity needed to run the entire monitoring pipeline.

3.1 Platform: Raspberry Pi 5

The Raspberry Pi 5 represents a significant architectural step forward from its predecessors, bringing a faster processor, dedicated PCIe connector, and improved GPIO performance. However, it also migrated its desktop environment to the Wayland compositing protocol — moving away from the legacy X11 display server — which would later become a critical source of software debugging work, as described in Section 5.

3.2 Real Hardware Components

Two physical output components were integrated via the Raspberry Pi 5 GPIO header using the BCM (Broadcom) pin numbering layout:

Component	Interface	BCM Pin Allocation	Purpose
HD44780 16×2 LCD	4-Bit Parallel Bus Mode	RS: 22, E: 18, D4: 25, D5: 24, D6: 23, D7: 12	Local visual readout of patient vital telemetry
Active Piezo Buzzer	Direct Digital GPIO Drive	Pin 17	Auditory alarm during threshold breach events

The HD44780 LCD operates in 4-bit parallel mode, requiring 6 GPIO lines total (RS, Enable, and 4 data lines D4–D7). This reduces the GPIO footprint compared to full 8-bit mode while maintaining full display functionality. The active buzzer on Pin 17 is driven by a simple digital HIGH/LOW signal — no PWM required — making it straightforward to trigger as an immediate alarm output.

3.3 Simulated Sensor Data

Rather than physically wiring biomedical sensors in this prototype phase, all three vital parameters were generated programmatically:

Vital Parameter	Simulation Method	Clinical Range Modelled	Purpose
Heart Rate	Random value generation within realistic BPM range	40 – 150 BPM	Cardiovascular monitoring, tachycardia detection
Body Temperature	Random value generation within clinical range	35°C – 42°C	Fever / hypothermia detection
Acceleration (3-axis)	Random G-force value generation	0 – 5 G	Fall detection / patient motion monitoring

This simulation approach is standard practice for prototype development. It allows the full data pipeline — from ingestion through to cloud transmission — to be built, tested, and validated without dependency on physical sensor hardware. The simulation script (`sim.py`) acts as a drop-in replacement that can be swapped for real sensor drivers once physical hardware integration is ready in a later phase.

4. The Hardware Struggle: SD Card Boot Failure

Before a single line of software could be written or tested, the project encountered its most time-consuming obstacle — a persistent hardware-level boot failure on the Raspberry Pi 5. This phase is documented here in full because it represents a significant and real part of the development timeline, and is a commonly underdocumented challenge in embedded system work.

4.1 Symptom: The Pink Screen

Every attempt to power on and boot the Raspberry Pi 5 resulted in one of the following failure states:

- A solid pink or rainbow-coloured screen appearing on the display immediately after power-on
- Various error messages appearing briefly on screen before the system halted
- Occasional partial boots that would freeze mid-way through the boot sequence
- Rare successful boots that worked normally, only for the problem to return on the next power cycle

The pink/rainbow screen on the Raspberry Pi 5 is a firmware-level diagnostic signal. It typically points to one of the following underlying causes:

Pink Screen Cause	Description
Corrupted SD card filesystem	The bootloader cannot read the required boot partition files from the card
Degraded SD card sectors	Physical wear or manufacturing defects causing intermittent read failures
Incompatible SD card speed class	The card cannot sustain the sequential read speed required for a reliable boot
Insufficient power supply	The Pi 5 draws more current than earlier models — underpowered supplies cause instability
Corrupted OS image write	The initial flashing of the OS image to the card was incomplete or corrupted

4.2 The Troubleshooting Period

The intermittent nature of the fault made diagnosis particularly difficult. The board would occasionally boot successfully — sometimes after waiting for extended periods — which initially suggested a power supply or thermal issue rather than a storage fault. This unpredictability led to a prolonged and frustrating troubleshooting period:

- Repeated power cycling in attempts to trigger a successful boot
- Waiting — sometimes for hours, occasionally spanning multiple days — hoping for a clean boot
- Attempting to work in the rare windows when the system came up successfully
- Re-flashing the OS image to the same SD card multiple times
- Checking power supply connections and cable integrity

The most challenging aspect of this phase was the **intermittency** of the failure. A completely dead system is straightforward to diagnose. A system that works occasionally — sometimes booting fine, other times showing the pink screen — is far harder to pin down. Days passed where meaningful development work was simply impossible, with time spent waiting and retrying rather than building.

4.3 Resolution: SD Card Replacement

After exhausting other diagnostic avenues, the SD card itself was identified as the root cause and replaced with a new card. The results were immediate and definitive — the Raspberry Pi 5 booted cleanly and consistently on every subsequent power cycle from that point onwards.

Key lesson: SD cards are a common and easily overlooked point of failure in Raspberry Pi projects, particularly on the Pi 5 which places higher I/O demands on the storage medium. Using a high-quality, Class 10 / U3 rated card from a reputable manufacturer is strongly recommended from the outset. For more demanding deployments, the Raspberry Pi Foundation recommends using an NVMe SSD via the Pi 5's PCIe connector as a more reliable alternative to SD card storage entirely.

With a stable hardware platform finally established, development of the monitoring software could begin in earnest.

5. Software Development & Debugging Challenges

With the hardware running reliably, the focus shifted fully to software. The development phase introduced two significant technical challenges that required substantial debugging and architectural rethinking before the system operated correctly.

5.1 Challenge A: The Blank Screen — Display Driver Collision

Symptom

During early visualization attempts using Python's matplotlib plotting library, one of two failure modes consistently appeared: either the application threw a critical display server initialization exception immediately on launch, or it opened a completely blank, unresponsive window with no plot content rendered.

Root Cause

The Raspberry Pi 5 ships with its desktop environment built on the Wayland compositing protocol — a modern replacement for the older X11 (X Window System) display server. This is a deliberate architectural change by the Raspberry Pi Foundation, but it introduced an immediate compatibility problem with the default matplotlib configuration.

By default, matplotlib attempts to use the Qt graphics engine (the QtAgg backend) to render plot windows. Qt's display abstraction layer (QPA — Qt Platform Abstraction) uses a platform plugin to connect to the underlying display server. On a Wayland-based system, this resulted in a

QT_QPA_PLATFORM environment mismatch — Qt was attempting to connect to an X11 display session that no longer existed as the primary compositor, causing the entire rendering pipeline to fail.

Resolution

The fix required switching matplotlib away from the Qt backend before any plotting code was executed. The TkAgg backend — which uses Python's built-in Tkinter canvas framework — does not share the same Wayland/X11 dependency and renders correctly on the Raspberry Pi 5. This was applied as a single line at the very top of the plotting script, before any matplotlib imports:

```
import matplotlib matplotlib.use('TkAgg') # Must be called before importing
matplotlib.pyplot import matplotlib.pyplot as plt
```

This single-line fix bypassed the entire Wayland/Qt compositing conflict and restored full plotting functionality. The key takeaway is that hardware platform migrations — even within the same product family — can silently break software assumptions that worked perfectly on the previous generation.

5.2 Challenge B: CSV Race Conditions & Memory Leaks

Architecture Overview

The data pipeline was designed around a shared CSV file (`sensor_data.csv`) acting as a lightweight message broker between three independently running scripts:

- `sim.py` — continuously generates simulated sensor values and writes them to the CSV
- `medical_system.py` — reads the latest values to drive the LCD, buzzer, and Blynk cloud sync
- `live_plot.py` — reads a moving window of recent values to render the live plots

Each script runs concurrently in separate terminal sessions, all accessing the same file simultaneously. This architecture introduced two distinct problems.

Problem 1: File Zeroing Race Condition

In the initial implementation, `sim.py` used write mode ('w') to open the CSV on every loop iteration — erasing and rewriting the entire file from scratch each time. This created regular windows where the file was completely empty (0 bytes). The reading scripts, running on independent timers, frequently caught the file during exactly these empty moments, causing:

- The live plot to render flat lines (no data available to plot)
- The medical system to throw index errors when trying to read the 'latest' row
- Occasional crashes when the parser received an empty or half-written CSV row

Problem 2: Memory Leak in Animation Buffer

The live plotting script used matplotlib's FuncAnimation to update the plot on a continuous loop. Without explicit management of the data arrays being appended on each frame, the arrays grew without bound —

accumulating every data point since script launch. Over time this caused steadily increasing RAM consumption, eventually slowing the entire system.

Resolution: Append-and-Trim Pattern

Both problems were addressed together by restructuring the pipeline around an Append-and-Trim pattern. Rather than overwriting the file, `sim.py` was changed to always append new rows while enforcing a maximum file length of 50 rows:

```
with open(FILENAME, 'a', newline='') as f: # 'a' = append, never overwrites
    writer = csv.writer(f)
    writer.writerow([hr, tmp, acc, t_now, d_now]) # Trim
    file to last 50 rows to prevent unbounded growth with open(FILENAME, 'r') as f:
    lines = f.readlines()
    if len(lines) > 50:
        with open(FILENAME, 'w') as f:
            f.writelines(lines[-50:])
```

On the reading side, defensive parsing was added to protect against any frame caught mid-write:

```
try:
    with open('sensor_data.csv', 'r') as f:
        lines = list(csv.reader(f))
        if len(lines) < 2:
            return # File is empty or header-only - skip this frame
        recent = lines[-20:] # Use only the last 20 rows (moving window)
except (ValueError, IndexError, csv.Error):
    pass # Silently discard corrupted or half-written frames
```

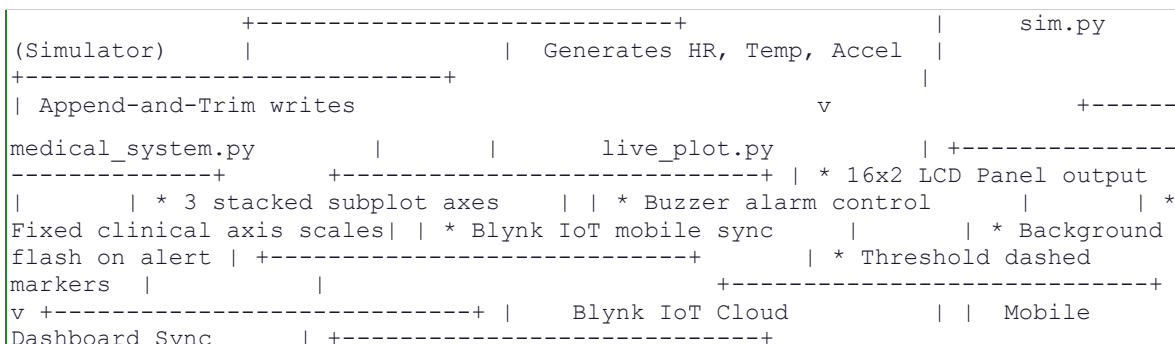
By combining append-only writes, file size enforcement, a moving-window read strategy, and defensive exception handling, both the race condition and the memory leak were eliminated. The system could then run for extended periods without degradation.

6. Final System Architecture

With hardware stable and software bugs resolved, the final architecture divides the monitoring workload across three independent scripts running concurrently in separate terminal sessions. Each script has a clearly defined responsibility and communicates exclusively through the shared CSV broker.

6.1 Data Flow Topology

The complete data flow is structured as a one-way pipeline from simulation through to cloud output, with two parallel consumers branching off the shared CSV:



6.2 Script Responsibilities

Script	Role	Key Outputs
sim.py	Sensor simulation engine — generates realistic random values for all three vitals on a timed loop	Appends rows to sensor_data.csv
medical_system.py	Real-time clinical processor — reads latest data, drives hardware outputs and cloud sync	LCD display, buzzer alarm, Blynk IoT dashboard
live_plot.py	Visual monitoring interface — renders 3-panel live plot with threshold indicators and alert flashing	Matplotlib animated window (TkAgg backend)

6.3 Visualization Strategy & Threshold Flags

The live plotting interface renders three vertically stacked subplots, one per vital parameter. All axes are fixed to medically meaningful ranges rather than auto-scaling, which would compress and mask clinically significant variations:

Parameter	Axis Range	Alarm Threshold	Alert Behaviour
Heart Rate	40 – 150 BPM	100 BPM	Plot background turns red when latest value exceeds 100 BPM
Body Temperature	35°C – 42°C	38.5°C	Plot background turns red when latest value exceeds 38.5°C
Acceleration	0 – 5 G	2.0 G	Plot background turns red when latest value exceeds 2.0 G

The threshold breach detection and visual alert is implemented as a conditional background colour switch on each subplot axis:

```
ax1.set_facecolor('#ffcccc' if hrs[-1] > HR_LIMIT else 'white') # Heart rate alert
ax2.set_facecolor('#ffcccc' if tmps[-1] > TMP_LIMIT else 'white') # Temperature alert
ax3.set_facecolor('#ffcccc' if accs[-1] > ACC_LIMIT else 'white') # Acceleration alert
```

Each subplot also displays a dashed horizontal line at the threshold value, providing a persistent visual reference for clinical personnel even when values are within normal range.

7. Conclusion

This project successfully delivered a working end-to-end prototype of a real-time patient monitoring system on the Raspberry Pi 5. The journey from initial hardware setup to a cloud-connected monitoring station involved overcoming real and significant challenges at both the hardware and software levels.

7.1 Key Challenges Overcome

Challenge	Category	Resolution
SD card boot failure — pink screen, days of intermittent boots	Hardware	Replaced SD card with a high-quality Class 10/U3 card
Wayland/Qt display driver mismatch — blank matplotlib windows	Software	Switched to TkAgg backend before matplotlib import
CSV race condition — file zeroed during concurrent read/write	Software	Implemented append-and-trim write pattern
Memory leak in animation buffer — unbounded array growth	Software	Applied moving-window (last 20 samples) read strategy
Unstable plot rendering — flat lines on data starvation	Software	Added defensive parsing with graceful frame discard

7.2 System Capabilities Delivered

- Real-time vital signs simulation (Heart Rate, Temperature, Acceleration)
- Local hardware output: 16×2 LCD display with live vital readings
- Auditory alarm buzzer triggered on threshold breach
- Live 3-panel matplotlib visualization with fixed clinical axis scaling
- Threshold-based alert system with visual background flash on each subplot
- Remote mobile monitoring via Blynk IoT cloud dashboard

7.3 Next Steps

The current prototype establishes a solid foundation for the following natural extensions:

- Replace the simulation engine with real physical sensors (MAX30102 for HR/SpO2, DS18B20 for temperature, MPU6050 for acceleration)
- Migrate from SD card storage to NVMe SSD via the Raspberry Pi 5 PCIe connector for improved I/O reliability
- Implement a proper inter-process communication mechanism (e.g., MQTT or shared memory) to replace the CSV file broker
- Explore deployment on a dedicated edge device with UPS battery backup for real clinical environment use